MIT 8103: ADVANCED DATABASE SYSTEMS

Name: Aisha Bello

Student ID: 301911151

Case Study: University Student Result Portal
GitHub Repository: https://github.com/abello1151-beep/mit-8103-student-result-portal
Academic Session: 2026/2027
Semester: First Semester

1. Introduction

This portfolio presents the design and practical exploration of a University Student Result Portal. The system manages student information, course details, registrations, and examination results. The project uses MySQL for relational database operations and MongoDB query examples to explore a NoSQL data model.

2. Portfolio 1: Database Design and Modelling

A relational database was designed using four tables: Students, Courses, Registrations, and Results. Primary keys identify records, while foreign keys establish relationships between students, courses, registrations, and results.

Constraints were included to enforce valid data, including unique student email addresses, unique course codes, and scores between 0 and 100. An ER diagram was created to illustrate the relationships. The design separates related information into tables to reduce unnecessary duplication and improve data integrity.

The database was organised to follow the First, Second and Third Normal Forms (1NF, 2NF and 3NF). Each table stores information about a specific subject, and relationships between tables are managed using primary and foreign keys. This helps reduce data duplication and update anomalies.

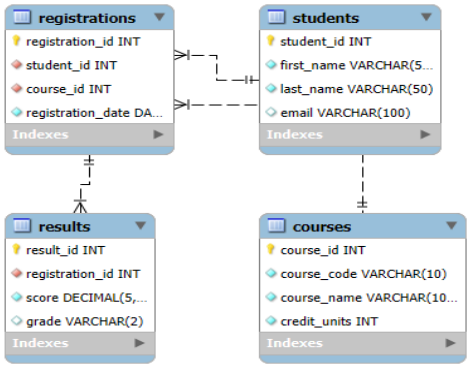


Figure 1: Entity Relationship Diagram showing the relationships between Students, Courses, Registrations and Results.

3. Portfolio 2: Query Processing and Optimisation

SQL queries were used to retrieve student results by joining the four tables. The project also explored query execution plans using MySQL's EXPLAIN statement.

An index named idx_student_last_name was created on the last_name column. Execution plans were examined to understand table scans and indexed lookups. Screenshots of query results and execution-plan evidence were saved in the GitHub repository.

The EXPLAIN results showed that a search without a suitable index used a table scan, while the surname search used the created index. This demonstrated how indexing can improve data retrieval.

Figure 2: SQL JOIN query showing student names, registered courses, scores and grades.

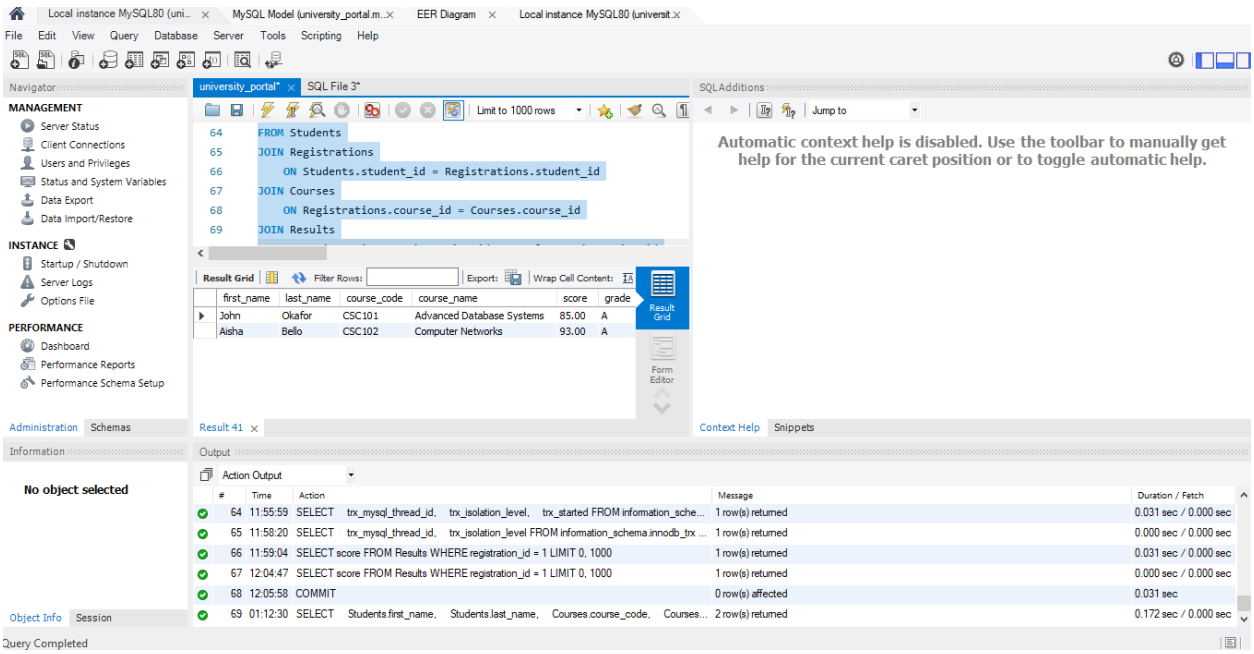
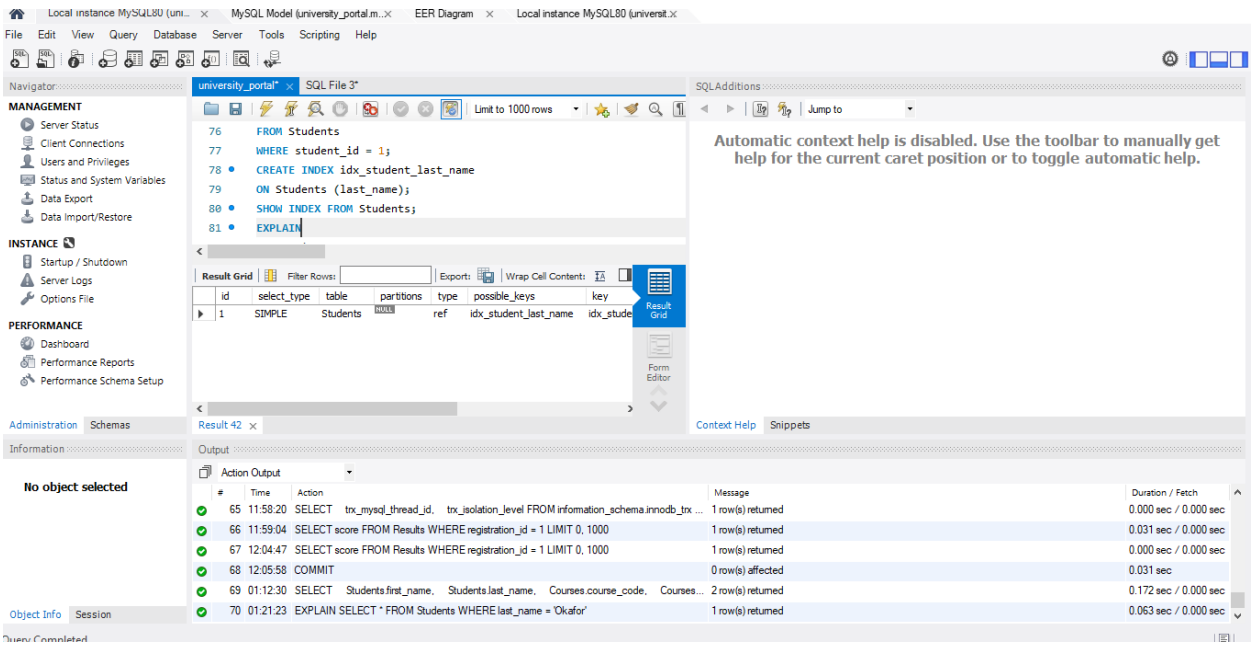


Figure 3: MySQL EXPLAIN output showing the indexed search on the Students table.



4. Portfolio 3: Transactions and Concurrency

Transactions were explored using START TRANSACTION, COMMIT, and ROLLBACK. A rollback test demonstrated that an uncommitted score change could be cancelled, while a committed change persisted. A two-session experiment was performed using the REPEATABLE READ isolation level. One session continued to see the original score of 85 while another session changed and committed the score as 75. After the first session committed, the updated value became visible to subsequent reads. The score was then restored to 85. This exercise demonstrated snapshot behavior under the tested isolation level.

Practical evidence: Screenshots of the COMMIT, ROLLBACK, and final score verification tests are provided. The two-session concurrency experiment is documented in the project SQL file.

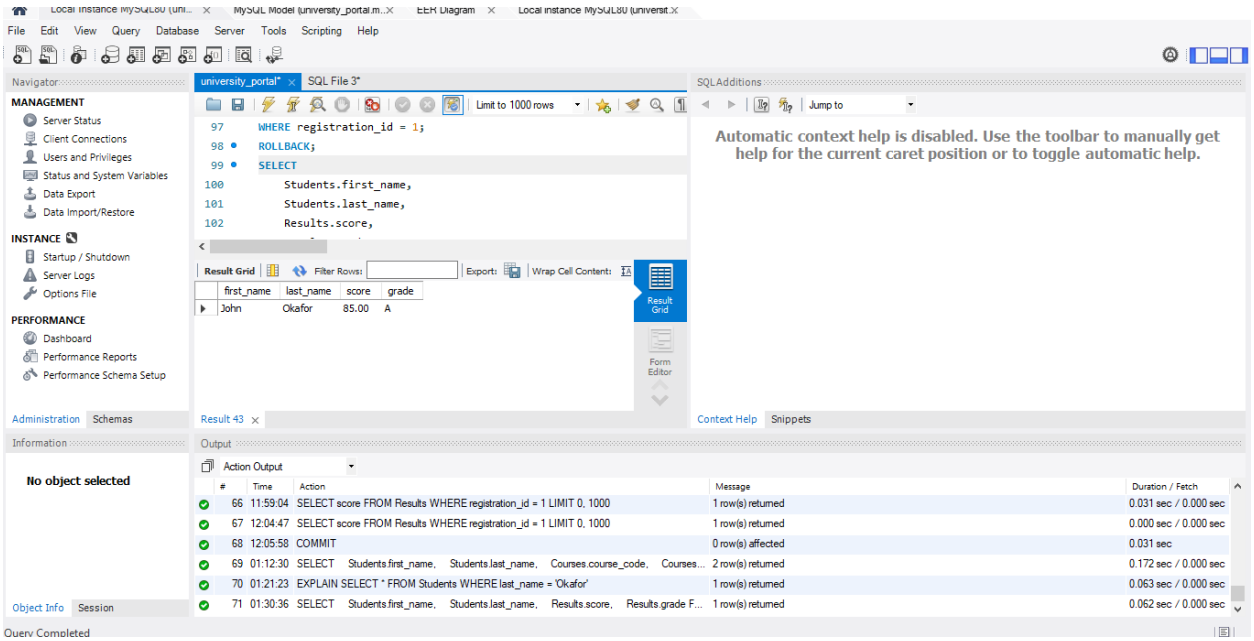


Figure 4: Evidence of the COMMIT operation, confirming that the updated student result was saved.

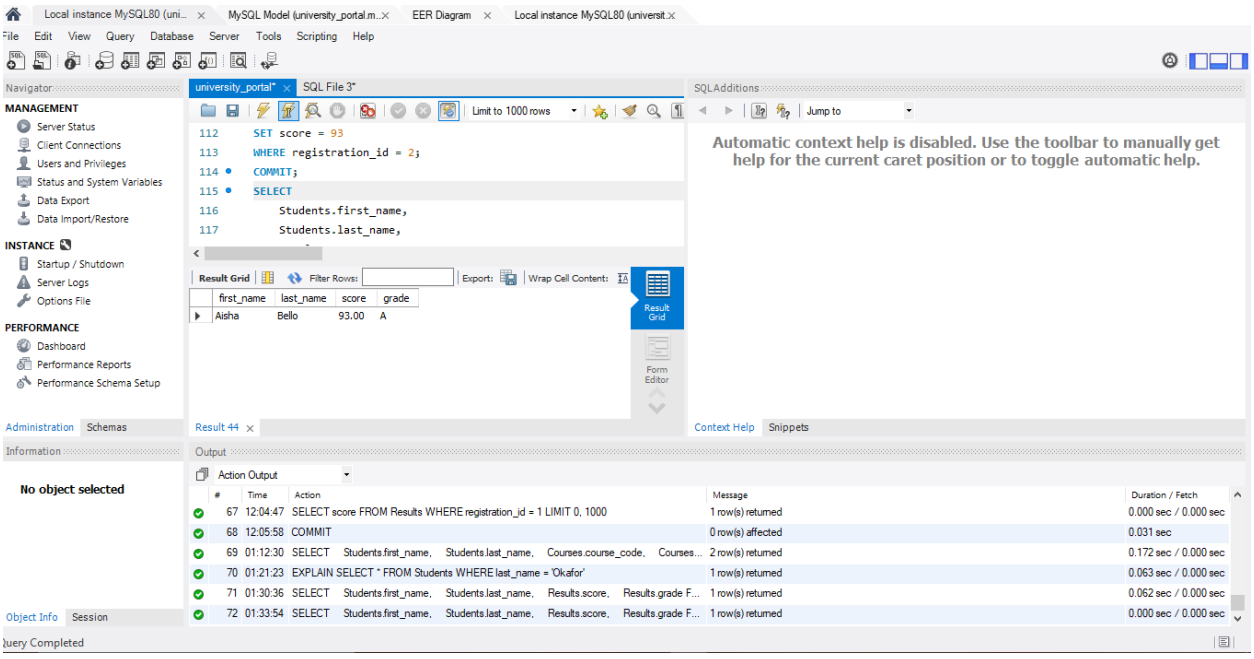
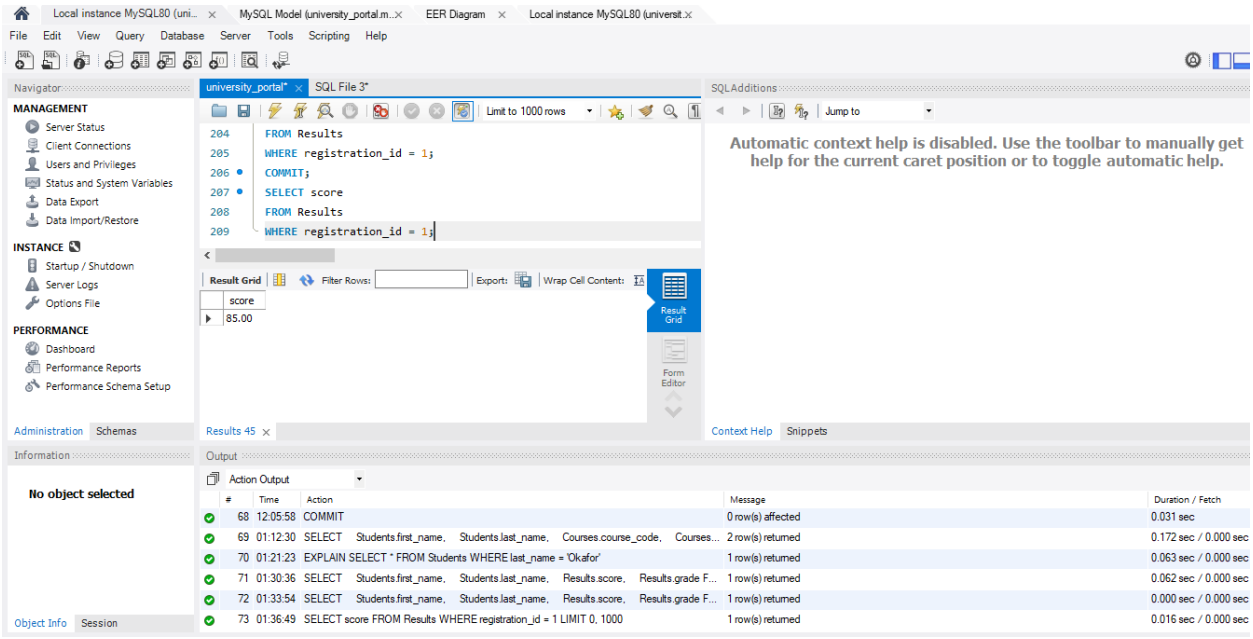


Figure 5: Final verification showing that the student's score was restored to 85.



5. Portfolio 4: NoSQL and Advanced Data Models

MongoDB-style document queries were practiced using Mongo Playground. Sample documents represented student names, course codes, scores, and grades. Queries were used to retrieve all records, find an individual student, filter scores above 90, and sort results by score.

The exercise illustrated how a document-oriented model can represent student result information. Mongo Playground was used for query practice; this work did not constitute a persistent MongoDB deployment.

A document model is suitable for this type of system because related student result details can be stored together in a flexible document. This can make retrieving a student's course and result information convenient.

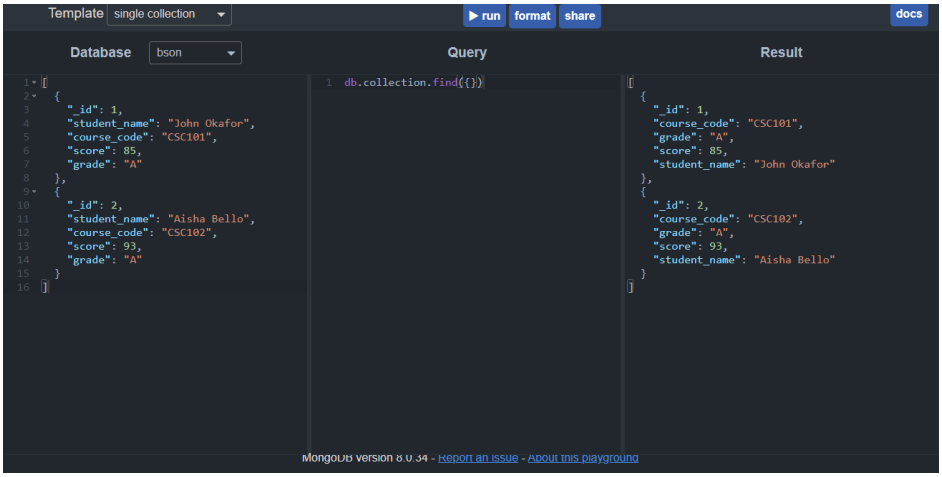


Figure 6: MongoDB document records showing the student names, course codes, scores and grades.



Figure 7: Query retrieving an individual student record.



Figure 8: Query filtering student records with scores greater than 90.

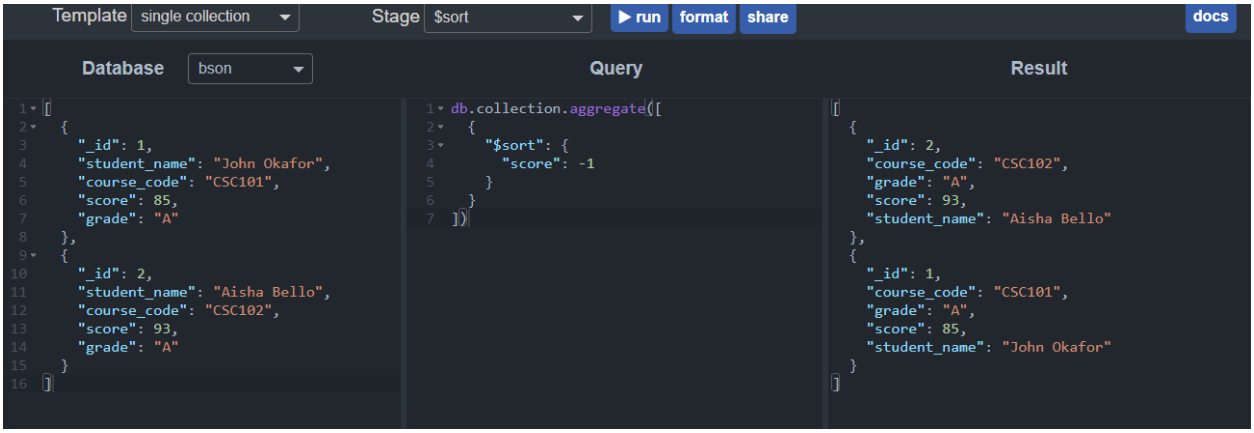


Figure 9: Aggregation query sorting student records by score in descending order.

6. Portfolio 5: Distributed and Cloud Database

The project considered how a cloud-hosted database could support remote access to the student portal. Replication could improve availability, while sharding could help distribute a growing volume of records. Consistency, security, backups, internet reliability, and cost were also considered.

This portfolio is a conceptual discussion. No actual cloud deployment, replication configuration, or sharding experiment was completed.

7. Reflection and Conclusion

This project improved my understanding of relational database design, SQL queries, indexing, transactions, concurrency, and document-oriented queries. I also learned the importance of testing database operations and keeping screenshots and scripts as evidence.

The practical exercises helped me understand how database constraints and transaction controls protect data integrity. Further work could include deploying a persistent NoSQL database, testing a cloud database, and measuring query performance with a larger dataset.

8. References

- MIT 8103 Advanced Database Systems, Individual Practical Database Portfolio assignment brief, 2026/2027 session.
- MySQL Documentation, https://dev.mysql.com/doc/
- MongoDB Documentation, https://www.mongodb.com/docs/

- Mongo Playground, https://mongoplayground.net/

9. AI-Use Declaration

ChatGPT was used to assist with organizing the report, explaining database concepts, and improving the presentation of project documentation. The SQL queries, database operations, and screenshots were developed and tested during the practical exercises.